Automating with Bash

NUnix · L3 MIAGE · CM3 · 90 minutes

ISTIC · University of Rennes 2026–2027



Louis Ledoux

Writing Bash scripts

Université de Rennes
Louis Ledoux

A script saves commands so you can run them again.
Arguments let you use the same script with different inputs.

1. Read arguments and store values in variables.
2. Use conditions, loops and functions.
3. Handle errors and test the script.

1. Scripts, arguments and variables

How can one script work with different inputs?

Bash: Bourne again

Université de Rennes
Louis Ledoux

Bash = Bourne-Again SHell.

The name puns on **Stephen Bourne**, author of the earlier Unix shell `sh`.

We use Bash syntax in this course.
`sh script.sh` may select a different shell with fewer features.

A first script

Université de Rennes
Louis Ledoux

```
#!/bin/bash
# hello.sh
name="MIAGE"
echo "Hello, $name"
```

- The first line selects an interpreter for direct execution.
- # begins a comment when used in this position.
- `.sh` is a naming convention, not a requirement.

Two ways to run a script

Université de Rennes
Louis Ledoux

Invoke Bash

```
bash hello.sh
```

Bash reads the file.
It needs **read** access to it.

Execute the file

```
chmod u+x hello.sh
./hello.sh
```

Requires **execute** permission and a usable interpreter.

The shebang: selecting an interpreter

Université de Rennes
Louis Ledoux

```
#!/bin/bash
printf 'Hello, MIAGE\n'
```

On Linux, direct execution of `./hello.sh` uses the interpreter named after `#!`.

The extension `.sh` does not choose Bash.
The first line does. We call it the **shebang**.

One symbol, several names

Université de Rennes
Louis Ledoux

Symbol	Names you may hear
#	hash, number sign; pound sign in US usage
#	French: **croisillon**, also **carré**
♯	The musical sharp sign (*dièse*)

#!: **hash** followed by **bang**.
The symbol's name and its role depend on context.

Positional parameters

Université de Rennes
Louis Ledoux

A script uses these parameters to read its arguments:

```
bash arguments.sh "L3 MIAGE" Unix
```

Parameter	Meaning	Value for this call
`$0`	Script name	`arguments.sh`
`$1`	First argument	`L3 MIAGE`
`$2`	Second argument	`Unix`
`$#`	Number of arguments	2
`"$@"`	All arguments separately	`"L3 MIAGE" "Unix"`
`"$*"`	All arguments as one string	`"L3 MIAGE Unix"`

Passing arguments with spaces

Université de Rennes
Louis Ledoux

Use the **same script call**:

```
bash arguments.sh "L3 MIAGE" Unix
```

Inside the script, `printf` uses `%s` for a string and `\n` for a newline.
It repeats the format for each argument.

"$@": two arguments

```
printf '<%s>\n' "$@"
```

```
<L3 MIAGE>
<Unix>
```

"$*": one string

```
printf '<%s>\n' "$*"
```

```
<L3 MIAGE Unix>
```

Use "$@" to pass each argument on unchanged.

Assigning and reading a variable

Université de Rennes
Louis Ledoux

```
course="NUnix"
echo "$course"
course="Bash scripting"
echo "Today: $course"
```

Assignment: `name=value`, with **no spaces** around `=`.
Expansion: `$name` or `${name}`.

Quoted variable expansions

Université de Rennes
Louis Ledoux

```
filename="meeting notes.txt"
cat "$filename"
echo '${filename}'
echo "${filename}"
```

- Double quotes preserve the expanded value as one argument.
- Single quotes preserve the literal text `${filename}`.

Capturing command output

Université de Rennes
Louis Ledoux

```
directory=$(pwd)
message="$1 and $2"
echo "$directory"
echo "$message"
```

`$(command)` captures the command's **standard output**.
`$1` and `$2` are positional arguments, introduced earlier.

Printing values

Université de Rennes
Louis Ledoux

```
name="Alice"
echo "Hello, $name"
printf 'Hello, %s\n' "$name"
```

echo is convenient for simple messages.
`printf` gives explicit control over formatting and newlines.

Exporting a variable

Université de Rennes
Louis Ledoux

```
message="hello"
bash -c 'echo "$message"'
export message
bash -c 'echo "$message"'
```

The first child prints an empty line.
After `export`, the next child prints **hello**.

A child receives a copy

Université de Rennes
Louis Ledoux

```
export message="parent"
bash -c 'message="child"; echo "$message"'
echo "$message"
```

```
child
parent
```

2. Conditions and exit status

How does a script decide what to do next?

Testing a condition

Université de Rennes
Louis Ledoux

Does the directory `logs` exist? These two commands ask the same question:

```
test -d logs
[ -d logs ]
```

`-d` means **is a directory**. `[ ... ]` is another way to write `test`.

A test returns a status; it prints no answer.
`0`: the condition is true. `1`: the condition is false.

Comparing integers

Université de Rennes
Louis Ledoux

Compare **integers** with these operators inside `[ ... ]`:

Operator	Read it as	Mathematical symbol
`-eq`	Equal to	=
`-ne`	Not equal to	≠
`-lt`	Less than	<
`-le`	Less than or equal to	≤
`-gt`	Greater than	>
`-ge`	Greater than or equal to	≥

Read `[ "$count" -gt 10 ]` as **count is greater than 10**.

if and else

Université de Rennes
Louis Ledoux

```
count=12
if [ "$count" -gt 10 ]; then
    echo "More than ten"
else
    echo "Ten or fewer"
fi
```

A true test runs the `then` branch; a false test runs `else`. `fi` marks the end.

Spaces are required after `[` and before `]`.
Correct: `[ "$count" -gt 10 ]`
Incorrect: `["$count" -gt 10]`

More than two cases: elif

Université de Rennes
Louis Ledoux

```
count=10
if [ "$count" -gt 10 ]; then
    echo "More than ten"
elif [ "$count" -eq 10 ]; then
    echo "Exactly ten"
else
    echo "Fewer than ten"
fi
```

`elif` means **else if**. Try each test in order and run the first true branch.
Here, the output is `Exactly ten`.

The condition can be grep

Université de Rennes
Louis Ledoux

```
if grep ' ERROR ' app.log; then
    echo "At least one error was found"
else
    echo "No match, or grep could not read the file"
fi
```

`if` runs `then` when the command returns status **0**.
Any other status selects `else`. Here, `grep` succeeds when it finds a matching line.

Conditional command lists

Université de Rennes
Louis Ledoux

```
mkdir results && echo "Directory created"
test -d results || echo "Directory is missing"
```

- a && b: run b only when a succeeds.
- a || b: run b only when a fails.
- ! a: invert the success/failure result.

Which messages appear?

Université de Rennes
Louis Ledoux

```
true  || echo "A"
false || echo "B"
true  && echo "C"
false && echo "D"
```

```
B
C
```

Checkpoint: command status

Université de Rennes
Louis Ledoux

1. Status `0`: true or false?
2. Does `false; echo yes` print?
3. Does `false && echo yes` print?
4. Status of `false || true`?

1. True / success.
2. Yes. `;` always continues.
3. No.
4. `0`, from the final executed command, `true`.

Check the argument count

Université de Rennes
Louis Ledoux

Save as `count-lines.sh`:

```
#!/bin/bash
if [ "$#" -ne 1 ]; then
    printf 'Usage: %s FILE\n' "$0" >&2
    exit 2
fi
wc -l < "$1"
```

Check the argument count **before** using $1.
Print the usage message on **stderr**.

Calculating with integers

Université de Rennes
Louis Ledoux

```
count=3
echo "$((count + 2))"
echo "$count"
```

```
5
3
```

Bash replaces `$((count + 2))` with 5 before running echo.
The calculation gives 5; `count` still contains 3.

No decimal part: `$((5 / 2))` gives 2.

Updating a counter

Université de Rennes
Louis Ledoux

```
count=3
count=$((count + 1))
echo "$count"
```

```
4
```

1. `$((count + 1))` calculates `3 + 1`, giving 4.
2. `count=...` stores that result back in `count`.

A loop can repeat this assignment to count its iterations.

3. Loops

How can I repeat commands for every file or line?

for visits a list of words

Université de Rennes
Louis Ledoux

```
for group in L3info L3miage M1info; do
    echo "$group"
done
```

Each iteration assigns the next word to `group`, then runs the body.

The list can contain literal words, numbers, filenames or `"$@"`.

while repeats after a successful test

Université de Rennes
Louis Ledoux

```
i=1
while [ "$i" -lt 10 ]; do
    echo "$i"
    i=$((i + 1))
done
```

Test **before** each iteration.
The loop prints the integers **1 through 9**.

while can test a whole command list

Université de Rennes
Louis Ledoux

```
i=1
while
    echo "Checking i=$i"
    [ "$i" -le 3 ]
do
    echo "Body: $i"
    i=$((i + 1))
done
```

The **last command** before do decides whether the body runs.

The condition runs once more than the body

Université de Rennes
Louis Ledoux

```
Checking i=1
Body: 1
Checking i=2
Body: 2
Checking i=3
Body: 3
Checking i=4
```

- At **1, 2, 3**, the test succeeds.
- At **4**, the final test fails.
- The body does not run a fourth time.

Read a file line by line

Université de Rennes
Louis Ledoux

```
while IFS= read -r name; do
    printf 'Hello, %s\n' "$name"
done < names.txt
```

- read succeeds when it reads a complete line.
- At **EOF**, it fails and the loop stops.
- < feeds the file to the **whole loop**.

until repeats after a failed test

Université de Rennes
Louis Ledoux

```
i=1
until [ "$i" -ge 10 ]; do
    echo "$i"
    i=$((i + 1))
done
```

This also prints **1 through 9**.
The loop stops when the condition succeeds.

break, continue and exit

Université de Rennes
Louis Ledoux

Command	Effect
`continue`	Skip the rest of the current iteration
`break`	Leave the current loop
`exit n`	End the script with status `n`

After `break`, execution continues **after the loop**.

case selects a matching pattern

Université de Rennes
Louis Ledoux

```
read -r -p "Enter a command: " choice
case "$choice" in
    all|ALL)
        ls -la ;;
    list|LIST)
        ls -l ;;
    *)
        echo "Invalid choice" ;;
esac
```

`|` separates alternative patterns. `;;` ends a branch.

4. Functions and debugging

How can I reuse commands and find mistakes?

Defining and calling a function

Université de Rennes
Louis Ledoux

```
greet() {
    echo "Hello, $1"
}

greet "MIAGE"
```

- Define the function **before calling it**.
- Inside it, `$1` is its first argument.
- `return n` finishes the function with exit status `n`.

Check syntax and trace execution

Université de Rennes
Louis Ledoux

```
bash -n count-lines.sh
bash -x count-lines.sh notes.txt
```

- `-n`: parse the script without executing its commands.
- `-x`: execute and trace expanded commands on stderr.
- Check output **and** status on ordinary and failing inputs.

CM3 summary: Bash scripts

Université de Rennes
Louis Ledoux

1. Define inputs, outputs and failure statuses.
2. Quote variable and filename arguments.
3. Use command status for branches and loop conditions.
4. Test normal, empty and failing inputs.

Optional reference

More examples and a command reference

Command substitution

Université de Rennes
Louis Ledoux

```
echo "Current directory: $(pwd)"
```

The shell runs pwd and replaces `$(pwd)` with its **standard output**.

The older form is `` `pwd` ``.
We will write `$(pwd)` because nested commands are easier to read.

Newlines, semicolons and continuations

Université de Rennes
Louis Ledoux

```
echo "first"
echo "second"; echo "third"
echo "a longer message" \
     "continued on the next line"
```

- A newline or ; separates commands.
- A backslash immediately before a newline continues the line.

Reading a line

Université de Rennes
Louis Ledoux

```
read -r -p "Your name: " name
echo "Hello, $name"
```

- read reads from **standard input**.
- `-p` displays a prompt in an interactive terminal.
- `-r` preserves backslashes in the input.

Integer, readonly and array variables

Université de Rennes
Louis Ledoux

```
declare -i count=3
readonly course="NUnix"
declare -a words
words[0]="one"
words[1]="two"
echo "${words[0]}"
```

`unset count` removes a variable. A readonly variable cannot be reassigned or unset in that shell.

Familiar environment variables

Université de Rennes
Louis Ledoux

Variable	Meaning
`HOME`	Home directory
`USER`	Account name, usually set at login
`PWD`, `OLDPWD`	Current and previous working directories
`PATH`	Colon-separated command search directories
`SHELL`	Configured login shell, not necessarily the current shell
`TERM`	Terminal type
`HOSTNAME`	Host name in Bash
`MANPATH`	Manual search path, when explicitly set

Arithmetic with let and expr

Université de Rennes
Louis Ledoux

```
let 'count = 3 + 5'
echo "$count"
expr 3 + 5
expr 3 \* 5
```

- `let` evaluates an arithmetic expression in Bash.
- `expr` takes separate arguments, so its spaces matter.
- Escape * so the shell passes it to `expr` literally.

Bash arithmetic shortcut: ((...))

Université de Rennes
Louis Ledoux

```
count=3
((count = count + 1))
echo "$count"
```

```
4
```

`((count = count + 1))` tells Bash to calculate and update `count`. It prints nothing; `echo` displays the new value.

Here, it does the same update as `count=$((count + 1))`.

Bash tests with double brackets

Université de Rennes
Louis Ledoux

```
filename="report.txt"
if [[ "$filename" == *.txt ]]; then
    echo "Text file"
fi
```

`[[ ... ]]` is another form of test available in Bash.
Here, `== *.txt` checks whether the name **ends in `.txt`**.

With `[ ... ]`, string equality compares literal text; this Bash form also accepts patterns.

Spaces are still required: `[[ condition ]]`.

Comparing numbers and strings

Université de Rennes
Louis Ledoux

Relation	Integers with [	Strings with [
Equal	`-eq`	`=`
Different	`-ne`	`!=`
Less / greater	`-lt`, `-gt`	`\<`, `\>`
Less / greater or equal	`-le`, `-ge`	No dedicated operator

```
[ "$count" -gt 3 ]
[ "$name" = "Alice" ]
```

Testing strings and files

Université de Rennes
Louis Ledoux

Test	True when…
`-n "$value"`	The string is nonempty
`-z "$value"`	The string is empty
`-e "$path"`	The path exists
`-f "$path"`	It is a regular file
`-d "$path"`	It is a directory
`-s "$path"`	Its size is greater than zero
`-r`, `-w`, `-x`	Read, write or execute/search access is allowed

Checkpoint: arithmetic and tests

Université de Rennes
Louis Ledoux

1. Value of `$((5 / 2))`?
2. Is `n = 3` an assignment?
3. Status of `false`?
4. Does `-f` accept every existing directory?

1. 2 (integer division).
2. No. Use n=3.
3. 1
4. No. It tests for a regular file.

Checking access to a file

Université de Rennes
Louis Ledoux

```
#!/bin/bash
read -r -p "Enter a filename: " filename
if [ ! -w "$filename" ]; then
    echo "File is not writable"
    exit 1
elif [ ! -r "$filename" ]; then
    echo "File is not readable"
    exit 1
else
    echo "File is writable and readable"
fi
```

Comparing two files

Université de Rennes
Louis Ledoux

```
diff "$1" "$2" > diff.out
status=$?
if [ "$status" -eq 0 ]; then
    echo "Files are identical"
elif [ "$status" -eq 1 ]; then
    cat diff.out
else
    echo "Comparison failed" >&2
fi
rm -f diff.out
```

Copying a known list of files

Université de Rennes
Louis Ledoux

```
#!/bin/bash
destination=$1
for file in f1 f2 f3; do
    cp "$file" "$destination/$file"
done
```

Call with an **existing destination directory** as the first argument.

A loop can feed a pipeline

Université de Rennes
Louis Ledoux

```
for number in 6 3 1 2; do
    echo "$number"
done | sort -n
```

```
1
2
3
6
```

Repeating until interrupted

Université de Rennes
Louis Ledoux

```
while true; do
    echo "alive..."
    sleep 3
done
```

`true` always succeeds.
Use **Ctrl+C** to interrupt this foreground loop.

Skipping 4, stopping at 7

Université de Rennes
Louis Ledoux

```
for x in 1 2 3 4 5 6 7 8 9; do
    if [ "$x" -eq 4 ]; then
        continue
    fi
    echo "$x"
    if [ "$x" -eq 7 ]; then
        break
    fi
done
echo "After the loop: x=$x"
```

The loop's output

Université de Rennes
Louis Ledoux

```
1
2
3
5
6
7
After the loop: x=7
```

- At **4**, `continue` skips `echo`.
- At **7**, `echo` runs before `break`.
- The final message runs after the loop.

Function scope

Université de Rennes
Louis Ledoux

```
outside="a global variable"
mine() {
    local inside="this is local"
    echo "$outside"
    echo "$inside"
    outside="a global with new value"
}
echo "$outside"
mine
echo "$outside"
echo "${inside-unset}"
```

What remains after the call

Université de Rennes
Louis Ledoux

```
a global variable
a global variable
this is local
a global with new value
unset
```

`outside` changed.
The function's **local variable** is no longer in scope.

Builtins: shell and process control

Université de Rennes
Louis Ledoux

Builtin	Use
`cd`, `pwd`	Change / report the current directory
`history`	Display command history
`exec`	Replace the shell with another program
`exit`	End the current shell
`kill`	Send a signal
`trap`	Arrange a response to a signal or shell event

Builtins: data and parameters

Université de Rennes
Louis Ledoux

Builtin	Use
`declare`, `readonly`, `local`	Define variable attributes or scope
`export`, `set`	Manage the environment / shell settings
`echo`, `read`	Print output / read input
`let`, `test`	Arithmetic / condition tests
`shift`	Discard leading positional parameters
`return`	End a function and report its status

Reading a small script

Université de Rennes
Louis Ledoux

```
#!/bin/bash
for filename in "$@"; do
    if [ -f "$filename" ]; then
        wc -l "$filename"
    fi
done
```

For each argument that names a regular file, print its line count.